

Exploratory Data Analysis (EDA)

A Complete, Cell-by-Cell Code Walkthrough

Univariate · Bivariate · Multivariate Analysis using Python

Prepared by: Sahajahanur Rahman

Email: connectingsrl@gmail.com | GitHub: Sahajahanur

Datasets: [cleaned_customer_sales.csv](#) · [fake_or_real_news.csv](#) · [weather.csv](#) · [daily_sales_data.csv](#)

1. Introduction

Exploratory Data Analysis (EDA) is the process of investigating, summarizing, visualizing, and cleaning a dataset to understand its structure, patterns, relationships, and problems before building models or making decisions. Performing EDA correctly is the foundation of accurate analytics — no matter how advanced a model or analytics technique is applied afterwards, if the data itself isn't understood, the resulting insights will not be accurate.

EDA is broken down into three core types of analysis, based on the number of features being analyzed at once:

- **Univariate Analysis** — analyzing a single feature at a time (its distribution, spread, and central tendency).
- **Bivariate Analysis** — analyzing the relationship between exactly two features.
- **Multivariate Analysis** — analyzing relationships among more than two features, and how they jointly influence a target variable.

This report walks through every code cell used in the accompanying notebook (**EDA.ipynb**), explaining what each line does, why it's used, and what insight it produces — across four datasets chosen specifically to demonstrate numeric, categorical, datetime, text, and time-series analysis in their proper context.

2. Data Preparation & Type Audit

Before any analysis, the dataset's column types must be identified — numeric, categorical, datetime, or text — since each type is analyzed differently. The customer sales dataset (**cleaned_customer_sales.csv**) is loaded first.

2.1 Load the dataset

```
import pandas as pd

df = pd.read_csv('cleaned_customer_sales.csv')
df.head()
```

Explanation: pandas is imported and the CSV is read into a DataFrame. `.head()` previews the first 5 rows to get a quick sense of the columns and values.

2.2 Check column data types

```
df.dtypes
```

Insight: CustomerID, Name, Gender, City are 'object' (text) type; Age is integer; Purchase_Amount and Feedback_Score are numeric; Signup_Date and Last_Purchase_Date show up as 'object' even though they represent dates — this means they must be explicitly converted to a proper datetime type before any time-based analysis can be done.

2.3 Convert date columns to datetime

```
# changing date columns to datetime
df['Signup_Date'] = pd.to_datetime(df['Signup_Date'])
df['Last_Purchase_Date'] = pd.to_datetime(df['Last_Purchase_Date'])
```

Explanation: `pd.to_datetime()` converts the text-based date strings into pandas' native `datetime64` type, which unlocks date-specific operations like `.dt.month`, `.dt.year`, subtraction between dates, resampling, and time series plotting.

```
df.dtypes
```

Insight: Re-checking `dtypes` confirms both columns are now `datetime64[ns]`, ready for time-based analysis.

3. Univariate Analysis

Univariate analysis studies one variable at a time to understand its distribution, central tendency (mean/median), and spread. The technique used depends entirely on the variable's type: numeric, categorical, datetime, or text.

3.1 Numeric Variables — Summary Statistics

The three numeric columns in the customer sales dataset are Age, Purchase_Amount, and Feedback_Score.

```
df.describe().T
```

Explanation: `.describe()` returns count, mean, std, min, 25%/50%/75% quartiles, and max for every numeric column. `.T` transposes the table so each row is a column and each column is a statistic — easier to read for many columns.

```
# summary statistics for numerical column
numeric_cols = ['Age', 'Purchase_Amount', 'Feedback_Score']

summary = df[numeric_cols].describe().T
summary['median'] = df[numeric_cols].median()
summary['IQR'] = summary['75%'] - summary['25%']
summary['skewness'] = df[numeric_cols].skew()
print("Numeric Summary Statistics:\n")
summary
```

Explanation: `.describe()` alone doesn't include median, IQR (Interquartile Range), or skewness — so these are added manually. Median is the true 'center' of the data (more robust to outliers than mean). IQR = 75th percentile – 25th percentile, used to detect the spread of the middle 50% of data and identify outliers. Skewness measures whether the distribution leans left or right of a normal (symmetric) distribution.

Insight: Purchase_Amount has a wide standard deviation (~14,000-15,000 range) and a 75th percentile around 35,000, indicating the spread is fairly wide but without extreme outliers pulling the distribution.

3.2 Numeric Variables — Visual Distribution Check

```
import matplotlib.pyplot as plt
import seaborn as sns
# histogram, boxplot, density plot for numerical columns
for col in numeric_cols:
    plt.figure(figsize=(12,4))

    plt.subplot(1,3,1)
    sns.histplot(df[col], kde=True)
    plt.title(f'Histogram of {col}')

    plt.subplot(1,3,2)
    sns.boxplot(y=df[col])
    plt.title(f'Boxplot of {col}')

    plt.subplot(1,3,3)
    sns.kdeplot(df[col], fill=True)
    plt.title(f'Density Plot of {col}')

plt.tight_layout()
plt.show()
```

Explanation: For each numeric column, three plots are generated side by side in one figure using `plt.subplot(1,3,n)`: a **histogram** (with KDE overlay) to see the frequency distribution and check for multiple peaks (multimodality); a **boxplot** to visually spot outliers (points beyond the whiskers) and see quartile spread; and a **density plot** (KDE) to see the smoothed shape of the distribution.

Insight: All three numeric columns show a single peak (no multimodality). `Feedback_Score` shows a slightly longer tail but not enough to be classified as an outlier — boxplots confirm no significant outlier points for any of the three columns.

3.3 Categorical Variables — Frequency Counts

```
# Frequency count for categorical columns
cat_cols = ['Gender', 'City']

for index, col in enumerate(cat_cols):
    print(f"\n Frequency counts for {col}:\n", df[col].value_counts())
    plt.figure(figsize=(6,4))
    sns.countplot(x=col, data=df, order=df[col].value_counts().index)
    plt.title(f'Bar Chart of {col}')
    plt.show()
```

Explanation: `.value_counts()` returns how many records fall into each category — this is the 'frequency count'. `sns.countplot()` visualizes the same thing as a bar chart, ordered from the most to least frequent category, so rare categories (low-frequency levels) and cardinality (number of unique categories) can be spotted at a glance.

Insight: Gender has a fairly even Male/Female split, and City has 4 categories (low cardinality) with Kolkata having the highest count — but no category is so rare that it should be considered for removal.

3.4 Datetime Variables — Range, Gaps & Granularity

```
date_cols = ['Signup_Date', 'Last_Purchase_Date']

for col in date_cols:
    print(f"\n Date Range for {col}: {df[col].min()} to {df[col].max()}")
    print(f"Time span: {(df[col].max() - df[col].min()).days} days")

# Month distribution
plt.figure(figsize = (12,4))
df['Signup_Month'] = df['Signup_Date'].dt.month_name()
sns.countplot(x='Signup_Month', data=df)
plt.title('Signup Month Distribution')
plt.show()
```

Explanation: `.min()/max()` find the earliest and latest date to establish the **date range**. Subtracting them and taking `.days` gives the total **time span** covered by the data. `.dt.month_name()` extracts the month name from each date so a count plot can show whether signups are evenly spread across the year or concentrated in specific months.

Insight: `Signup_Date` spans about 5 years (Oct 2020 to Oct 2025) while `Last_Purchase_Date` only spans the most recent 365 days. Signup month distribution is close to uniform across all months, meaning signups happen consistently year-round.

3.5 Time Granularity Check

```
def time_granularity(series):
    diffs = series.sort_values().diff().dropna().dt.days
    return diffs.describe()
```

```
time_granularity(df['Last_Purchase_Date'])

df['Last_Purchase_Date'].sort_values().diff().value_counts()

time_granularity(df['Signup_Date'])

df['Signup_Date'].sort_values().diff().value_counts()
```

Explanation: The custom `time_granularity()` function sorts the date column, then computes the difference (in days) between each consecutive date using `.diff()`. This reveals whether the data is recorded daily, weekly, monthly, or with irregular gaps. `.value_counts()` on the differences shows exactly how many 0-day, 1-day, 2-day gaps (etc.) exist in the sorted sequence.

Insight: `Last_Purchase_Date` shows mostly 0-day and 1-day gaps → data granularity is daily. `Signup_Date` shows gaps up to 3 days in places, still confirming a day-level granularity with a few small gaps (likely no signups on certain days).

3.6 Time Series Trend, Seasonal Decomposition & Heatmap

```
df.groupby('Last_Purchase_Date')['Purchase_Amount'].sum().plot()
plt.show()
```

Explanation: Groups all purchases by date and sums the `Purchase_Amount` per day, then plots it as a line chart to visualize the overall trend and spot any spikes or anomalies over time.

```
from statsmodels.tsa.seasonal import seasonal_decompose

df_daily = df.groupby('Last_Purchase_Date')['Purchase_Amount'].sum()
result = seasonal_decompose(df_daily, model='additive', period=30)
result.plot().show()
```

Explanation: `seasonal_decompose()` splits the daily sales series into three components: **Trend** (long-term direction), **Seasonal** (repeating pattern, `period=30` days here), and **Residual** (leftover noise not explained by trend/seasonality). The 'additive' model is chosen because the series doesn't show increasing/multiplying magnitude over time.

Insight: The trend component is flat (no strong upward/downward direction), and the seasonal component shows a repeating pattern roughly every month to month-and-a-half, with a fair amount of residual noise.

```
df["day"] = df["Last_Purchase_Date"].dt.day
df["month"] = df["Last_Purchase_Date"].dt.month
heatmap_data = df.pivot_table(
    index="day",
    columns="month",
    values="Purchase_Amount",
    aggfunc="mean"
)
heatmap_data

# Plot heatmap
plt.figure(figsize=(12,12))
sns.heatmap(heatmap_data, cmap='YlGnBu', annot=True, fmt=".1f")
plt.xlabel("Month")
plt.ylabel("Day of Month")
plt.title("Purchase Heatmap: Day vs Month")
plt.show()
```

Explanation: Day and month are extracted from the date, then a pivot table is built with day-of-month as rows and month as columns, with the average `Purchase_Amount` as the value in each cell. A heatmap visualizes this grid — darker cells mean a higher average purchase amount on that day-of-month/month

combination.

Insight: The darkest cell appears around day 2 of month 10 (October), indicating that date historically had the highest average purchase amount. Some cells are blank because not every month has 29-31 days (e.g., no 30th/31st in February).

3.7 Text Variables — Preprocessing, Tokenization & Word Frequency

A separate dataset, **fake_or_real_news.csv**, is used here since text columns (title/text) are needed for this type of univariate analysis.

```
text_df = pd.read_csv('fake_or_real_news.csv')
text_df

import re
# Text preprocessing
def clean_text(text):
    text = text.lower()
    text = re.sub(r"[^a-z\s]", "", text)
    return text

text_df["clean_text"] = text_df["text"].apply(clean_text)
```

Explanation: Text preprocessing/cleaning is required before analysis. `.lower()` converts everything to lowercase so 'The' and 'the' are treated as the same word. The regex pattern `[^a-z\s]` matches any character that is **not** a lowercase letter or whitespace, and `re.sub()` replaces all such characters (punctuation, numbers, symbols) with nothing — effectively stripping them out.

```
text_df["tokens"] = text_df["clean_text"].apply(lambda x: x.split())
```

Explanation: `.split()` breaks the cleaned text into a list of individual words on whitespace — these individual words are called **tokens**.

```
text_df["token_count"] = text_df["tokens"].apply(len)
text_df
```

Explanation: `len()` applied to each token list gives the number of words (tokens) in that record — a new numeric feature representing text length.

```
print("Token Count Summary")
print(text_df["token_count"].describe())

# Histogram of text lengths
plt.figure()
plt.hist(text_df["token_count"])
plt.xlabel("Number of Words per Review")
plt.ylabel("Frequency")
plt.title("Histogram of Review Text Lengths")
plt.show()
```

Insight: Token count ranges from a minimum of 0 words to a maximum of over 14,000 words — a huge range, showing heavy right-skew and clear outliers. Most articles cluster around 1,800-1,900 words, with a long tail of much longer outlier articles.

```
# most common words
from collections import Counter
all_tokens = [token for tokens in text_df["tokens"] for token in tokens]
word_freq = Counter(all_tokens)

top_words = word_freq.most_common(10)
```

```
top_words_df = pd.DataFrame(top_words, columns=["word", "count"])

print("\nTop 10 Most Common Words")
print(top_words_df)
```

Explanation: A nested list comprehension flattens every token list across all rows into one giant list of words (`all_tokens`). `Counter` counts how often each word appears, and `.most_common(10)` returns the top 10 most frequent words.

Insight: Before cleaning stopwords, the top words are common filler words like 'the', 'to', 'and' — these don't carry any real meaning for analysis.

```
# after removing stopwords
from nltk.corpus import stopwords
stopword = set(stopwords.words())
all_tokens = [token for token in all_tokens if token not in stopword]
word_freq = Counter(all_tokens)

top_words = word_freq.most_common(10)
top_words_df = pd.DataFrame(top_words, columns=["word", "count"])

print("\nTop 10 Most Common Words")
print(top_words_df)

plt.figure()
plt.bar(top_words_df["word"], top_words_df["count"])
plt.xlabel("Words")
plt.ylabel("Frequency")
plt.title("Top 10 Most Common Words (after removing stopwords)")
plt.show()
```

Explanation: Stopwords are extremely common words ('the', 'is', 'and', etc.) that add no analytical value. NLTK's built-in stopword list is loaded as a set (for fast lookup), and every token that appears in that set is filtered out before recomputing the frequency count.

Insight: After removing stopwords, the true top words emerge — 'Trump', 'Clinton', 'state', 'president' — confirming this news dataset is predominantly about politics and the U.S. presidential election.

4. Bivariate Analysis

Bivariate analysis studies the relationship between exactly two variables. The technique depends on whether both variables are numeric, one is numeric and one categorical, both are categorical, or both represent a time series.

4.1 Numeric – Numeric: Correlation & Regression

A weather dataset (**weather.csv**) is used here since it has several numeric columns that are naturally related to each other.

```
weather_df = pd.read_csv('weather.csv')
weather_df

corr = weather_df.corr()

plt.figure(figsize=(5,4))
sns.heatmap(corr, annot=True, cmap='coolwarm', fmt='.2f')
plt.title('Correlation Heatmap')
plt.show()

# Pairwise scatter plots + regression line
sns.pairplot(weather_df, kind='reg', diag_kind='kde', corner=True,
plot_kws={'line_kws':{'color':'red'}})
plt.suptitle('Scatter and Regression Lines Between Numeric Variables', y=1.02)
plt.show()
```

Explanation: `.corr()` computes the Pearson correlation coefficient between every pair of numeric columns (ranges from -1 to +1). The heatmap visualizes this matrix with colors. `sns.pairplot(kind='reg')` then plots every pairwise scatter plot in the dataset with a fitted regression line (in red) on top, so both the correlation strength and the direction/shape of the relationship can be seen visually.

Insight: Temperature and Apparent Temperature show a very strong positive correlation (~0.9), forming an almost straight diagonal line in the scatter plot — meaning they move together almost identically. Wind Speed shows a declining trend against Temperature.

4.2 Numeric – Categorical: Grouped Boxplots & ANOVA

```
cat_cols = ['Gender', 'City']

for cat in cat_cols:
    for num in numeric_cols:
        plt.figure(figsize=(7,2))
        sns.boxplot(x=cat, y=num, data=df)
        plt.title(f'{num} by {cat}')
        plt.show()

print(f"\n Group Mean and Median of {num} by {cat}:")
print(df.groupby(cat)[num].agg(['mean', 'median', 'count']))
```

Explanation: For every combination of a categorical column (Gender, City) and numeric column (Age, Purchase_Amount, Feedback_Score), a boxplot is drawn with the category on the x-axis — this produces one boxplot per category, colored/grouped side by side, so distributions can be compared directly. `.groupby().agg()` then prints the exact mean, median, and count for each group.

Insight: Across all combinations, the mean/median values are very close between groups (e.g., Male vs Female, or across cities) — visually there is little difference in distribution shape or center.

```

from scipy import stats
anova_res = stats.f_oneway(*[group["Feedback_Score"].values for name, group in
df.groupby("City")])
print("ANOVA Test for Feedback_Score across Cities:")
print("F-statistic:", round(anova_res.statistic, 3), ", p-value:",
round(anova_res.pvalue, 3))

```

Explanation: A visual comparison isn't enough to confirm a real difference — a **one-way ANOVA test** checks if the difference in Feedback_Score means across the different City groups is *statistically* significant. `df.groupby('City')` splits the data into groups, and `stats.f_oneway()` compares all group means at once.

Insight: F-statistic = 0.998, p-value = 0.417. Since p-value > 0.05, there is **no statistically significant difference** in Feedback_Score across the different cities — the visual similarity is confirmed statistically.

4.3 Categorical – Categorical: Contingency Table & Chi-Square Test

```

contingency = pd.crosstab(df['Gender'], df['City'])
print("\n Contingency Table (Gender vs City):\n", contingency)

# Stacked Bar Chart
contingency.plot(kind='bar', stacked=True, figsize=(8,5))
plt.title('Stacked Bar Chart: Gender vs City')
plt.ylabel('Count')
plt.show()

# Chi-Square Test of Independence
chi2, p, dof, expected = stats.chi2_contingency(contingency)
print("\nChi-Square Test Results:")
print(f"Chi2 = {chi2:.3f}, p-value = {p:.3f}, Degrees of Freedom = {dof}")

if p < 0.05:
    print("Significant relationship between Gender and City")
else:
    print("No significant relationship between Gender and City")

```

Explanation: `pd.crosstab()` builds a **contingency table** — a cross-tabulated count of every Gender × City combination. This is visualized as a stacked bar chart. The **Chi-Square test of independence** (`stats.chi2_contingency()`) then formally checks whether Gender and City are statistically independent of each other, or whether one influences the distribution of the other.

Insight: The p-value from the Chi-Square test is greater than 0.05, meaning there is **no significant relationship** between Gender and City — the two categorical variables are statistically independent in this dataset.

4.4 Time Series Relationships: Lag Plots, ACF & PACF

A dedicated time series dataset (**daily_sales_data.csv**) with just two columns — date and sales — is used here.

```

sales_df = pd.read_csv('daily_sales_data.csv')
sales_df['date'] = pd.to_datetime(sales_df['date'])
sales_df

from pandas.plotting import lag_plot

plt.figure()
lag_plot(sales_df["sales"], lag=1)
plt.title("Lag Plot (Lag = 1 day)")

```

```
plt.show()

plt.figure()
lag_plot(sales_df["sales"], lag=7)
plt.title("Lag Plot (Lag = 7 days)")
plt.show()
```

Explanation: A **lag plot** plots each value in the series against the value from N periods earlier (lag=1 means yesterday vs today, lag=7 means a week ago vs today). If points cluster along a diagonal line, it means the series is correlated with its past — if points are scattered randomly, there's no such relationship.

Insight: Both the lag-1 and lag-7 plots show a completely scattered pattern with no diagonal trend — meaning daily sales values are not predictable from their own recent past.

```
from statsmodels.graphics.tsaplots import plot_acf

plt.figure()
plot_acf(sales_df["sales"], lags=30)
plt.title("Autocorrelation (ACF) of Daily Sales")
plt.show()
```

```
from statsmodels.graphics.tsaplots import plot_pacf

plt.figure()
plot_pacf(sales_df["sales"], lags=40, method="ywm")
plt.title("PACF - Daily Sales")
plt.show()
```

Explanation: **ACF (Autocorrelation Function)** measures how correlated the series is with its own past values at increasing lags (1, 2, 3... days). **PACF (Partial Autocorrelation Function)** measures the correlation at each lag *after removing* the effect of the shorter lags in between. These plots are typically used to identify the p, d, q parameters for ARIMA-style time series models.

Insight: Both ACF and PACF bars stay mostly within the confidence interval (not statistically significant) confirming the lag plot's finding — no meaningful autocorrelation in this daily sales series.

```
sales_df["rolling_7"] = sales_df["sales"].rolling(window=7).mean()
sales_df["rolling_14"] = sales_df["sales"].rolling(window=14).mean()

plt.figure()
plt.plot(sales_df.index, sales_df["sales"], label="Daily Sales")
plt.plot(sales_df.index, sales_df["rolling_7"], label="7-day Rolling Avg")
plt.plot(sales_df.index, sales_df["rolling_14"], label="14-day Rolling Avg")
plt.xlabel("Date")
plt.ylabel("Sales")
plt.title("Rolling Average Analysis")
plt.legend()
plt.show()
```

Explanation: `.rolling(window=N).mean()` computes a moving average over the last N days at every point in the series. This smooths out short-term day-to-day fluctuations/spikes so the underlying trend becomes easier to see. A 7-day window smooths weekly noise; a 14-day window smooths even more.

Insight: The rolling average lines flatten out the daily noise and confirm the series stays fairly flat/uniform over time, with no clear upward or downward trend.

5. Multivariate Analysis

Multivariate analysis studies how more than two variables interact together, and how they jointly influence a target variable — going beyond simple pairwise relationships.

5.1 Interaction Plots

```
# Example: Age + Gender -> Purchase Amount
plt.figure(figsize=(8,5))
sns.boxplot(
    data=df,
    x='Gender',
    y='Purchase_Amount',
    hue='Age',
    dodge=True
)
plt.title('Interaction Plot: Gender & Age vs Purchase Amount')
plt.show()
```

```
# Another example: City + Feedback -> Purchase Amount

plt.figure(figsize=(8,5))
sns.boxplot(
    data=df,
    x='City',
    y='Purchase_Amount',
    hue='Feedback_Score'
)
plt.title('Interaction: City & Feedback vs Purchase Amount')
plt.show()
```

Explanation: The hue parameter adds a third dimension of color to a boxplot — so instead of comparing just Gender vs Purchase_Amount, the color also shows how Age changes the picture. This reveals **interaction effects**: does the relationship between Gender and Purchase Amount change depending on Age group?

Insight: Older customers (darker colors) tend to have higher Purchase_Amount consistently across both Gender categories — confirming that age has a positive relationship with spending, regardless of gender.

5.2 Dimensionality Reduction with PCA

```
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
X = df[numeric_cols]

# Standardize
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# PCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

df['PCA1'] = X_pca[:,0]
df['PCA2'] = X_pca[:,1]

print("Explained Variance Ratio:", pca.explained_variance_ratio_)
```

Explanation: StandardScaler rescales all numeric columns to have mean 0 and standard deviation 1 — this is essential before PCA because it prevents columns with larger numeric ranges (like Purchase_Amount) from dominating the result. PCA(n_components=2) then compresses the 3 numeric columns down into 2 new 'principal components' (PCA1, PCA2) that capture as much of the original variance as possible, making it possible to visualize the multidimensional data in a simple 2D scatter plot.

```
plt.figure(figsize=(7,5))
sns.scatterplot(
    x='PCA1',
    y='PCA2',
    data=df,
    s=100
)
plt.title('PCA Projection (2D)')
plt.show()
```

Insight: The explained variance ratio shows how much of the original information is retained in these 2 new components — this 2D plot is what gets fed into the clustering step next.

5.3 Customer Segmentation with KMeans Clustering

```
# KMeans clustering
kmeans = KMeans(n_clusters=2, random_state=42)
df['Cluster'] = kmeans.fit_predict(X_pca)
plt.figure(figsize=(7,5))
sns.scatterplot(
    x='PCA1',
    y='PCA2',
    hue='Cluster',
    data=df,
    palette='Set1',
    s=100
)
plt.title('Customer Segments using KMeans')
plt.show()
```

Explanation: KMeans(n_clusters=2) is an unsupervised algorithm that groups data points into 2 clusters based on proximity in the PCA-reduced space, without being told any labels in advance. .fit_predict(X_pca) both trains the model and assigns each row a cluster label (0 or 1) in one step.

```
df['Cluster'].value_counts()
```

```
print(df.groupby('Cluster')[numeric_cols].mean())
```

Explanation: .value_counts() shows how many customers fall into each cluster. .groupby('Cluster').mean() then compares the average Age, Purchase_Amount, and Feedback_Score between the two clusters to interpret what each segment actually represents in business terms.

Insight: KMeans split customers into two segments — roughly 4,700 records in one cluster and 4,100 in the other. Age is almost identical between clusters, but **Purchase_Amount differs sharply** (~36,000 mean vs ~14,000 mean), and **Feedback_Score** also differs (~5 vs ~4). This means the clustering has effectively separated customers into a **high-value, high-satisfaction segment** and a **low-value, low-satisfaction segment** — a directly usable insight for targeted marketing or retention strategy.

6. Summary of Findings

Analysis	Dataset	Key Finding
Univariate (Numeric)	Customer Sales	No significant outliers; single-peak distributions across Age, Purchase_Amount, Feedback_Score.
Univariate (Categorical)	Customer Sales	Balanced Gender split; City has low cardinality (4 categories), no rare categories.
Univariate (Datetime)	Customer Sales	Daily granularity; signup data spans ~5 years, purchases span 1 year; peak avg. purchase in winter.
Univariate (Text)	Fake/Real News	Heavy right-skewed token counts; top words after stopword removal indicate political/social topics.
Bivariate (Numeric-Numeric)	Weather	Temperature and Apparent Temperature strongly correlated (~0.9).
Bivariate (Numeric-Categorical)	Customer Sales	ANOVA p-value = 0.417 (>0.05) → no significant Feedback_Score difference across Cities.
Bivariate (Categorical-Categorical)	Customer Sales	Chi-Square p-value > 0.05 → Gender and City are statistically independent.
Bivariate (Time Series)	Daily Sales	No meaningful autocorrelation at lag-1 or lag-7; series is largely random/flat.
Multivariate (PCA + KMeans)	Customer Sales	Two clear customer segments found, differentiated by Purchase_Amount and Feedback_Score.

Next Steps

- Extend text analysis with TF-IDF/n-grams instead of raw word frequency counts.
- Build a reusable function/class that auto-detects column types and runs this entire EDA pipeline on any new dataset.
- Add outlier treatment and feature engineering as a follow-up step, moving from EDA into model-ready data.
- Try UMAP/t-SNE alongside PCA to compare non-linear cluster structure.